

HDOC DAY-28

Title: Variable-Length 2D Arrays, Column-Major Input, and Linear Indexing Conversion

1. Problem Statement

1. Write a C program to:

- Read rows and columns, ensuring positive dimensions with input re-prompting.
- Declare a variable-length 2D integer array.
- Read elements column by column using nested loops (outer loop over columns, inner loop over rows).
- Print elements in row-major order using a single loop with linear index calculations.

2. Write a C program that reads elements into a variable-length 2D array, copies them into a 1D array in row-major order, and displays both arrays.

2. Algorithms

Algorithm 1: Program 1 (Column-Wise Input & Single-Loop Linear Access)

1. Prompt and validate user inputs for rows and cols (> 0) using do-while loops.
2. Declare variable-length 2D array `matrix[rows][cols]`.
3. For column $j = 0$ to $cols - 1$: for row $i = 0$ to $rows - 1$: read `matrix[i][j]`.
4. Set `total_elements = rows * cols`.
5. Loop $k = 0$ to $total_elements - 1$: calculate row $r = k / cols$ and column $c = k \pmod{cols}$, then print `matrix[r][c]`. Print a newline when $(k + 1) \pmod{cols} == 0$.

Algorithm 2: Program 2 (2D Array to 1D Array Copy)

1. Read rows and cols.
2. Declare `matrix[rows][cols]` and `array1D[rows * cols]`.
3. Read elements into `matrix[i][j]` using nested row-major loops.
4. Flatten 2D array to 1D array: `array1D[index++] = matrix[i][j]`.
5. Display both matrix (grid format) and array1D (linear array format).

3. Test Case Table

Tes t Cas e	Progra m	Descripti on / Inputs	Expect ed Output	Actual Output	Stat us
1	Prog 1	Input: rows=-2, then rows=2, cols=3	Error messa ge on negativ e, prompt s until positiv e	Validati on messa ge shown	Pass
2	Prog 1	Col-wise input for 2×3 : $C_0 =$ (10,20), C_1 (30,40), C_2 (50,60)	Row- Major Grid: Row 0: 10 30 50, Row 1: 20 40 60	Matche d row- major order	Pass

Tes t Cas e	Progra m	Descripti on / Inputs	Expect ed Output	Actual Output	Stat us
3	Prog 2	2 × 2 matrix: 1,2,3,4	2D: $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$, 1D: [1 2 3 4]	Matche d expect ed output s	Pass

4. Source Code Listing

Program 1 Source Code:

```
#include <stdio.h>

int main(void) {
    int rows, cols;

    do {
        printf("Enter positive number of rows: ");
        scanf("%d", &rows);
        if (rows <= 0) printf("Invalid input! Rows must be positive.\n");
    } while (rows <= 0);

    do {
        printf("Enter positive number of columns: ");
        scanf("%d", &cols);
        if (cols <= 0) printf("Invalid input! Columns must be positive.\n");
    } while (cols <= 0);

    int matrix[rows][cols];
```

```

printf("\nEnter elements column by column:\n");
for (int j = 0; j < cols; j++) {
    for (int i = 0; i < rows; i++) {
        printf("Element at Matrix[%d][%d]: ", i, j);
        scanf("%d", &matrix[i][j]);
    }
}

int total_elements = rows * cols;
printf("\nMatrix printed in Row-Major Order (using single loop & linear indexing):\n");
for (int k = 0; k < total_elements; k++) {
    int r = k / cols;
    int c = k % cols;
    printf("%d\t", matrix[r][c]);
    if ((k + 1) % cols == 0) printf("\n");
}

return 0;
}

```

Program 2 Source Code:

```

#include <stdio.h>

int main(void) {
    int rows, cols;
    printf("Enter number of rows: ");
    scanf("%d", &rows);
    printf("Enter number of columns: ");
    scanf("%d", &cols);

    if (rows <= 0 || cols <= 0) {
        printf("Error: Matrix dimensions must be positive integers.\n");
        return 1;
    }

    int matrix[rows][cols];
    int array1D[rows * cols];

```

```

printf("\nEnter 2D Array Elements:\n");
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        printf("Matrix[%d][%d]: ", i, j);
        scanf("%d", &matrix[i][j]);
    }
}

int index = 0;
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        array1D[index++] = matrix[i][j];
    }
}

printf("\n2D Array Matrix Representation:\n");
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        printf("%d\t", matrix[i][j]);
    }
    printf("\n");
}

printf("\nCopied 1D Array Representation:\n[ ");
for (int k = 0; k < rows * cols; k++) {
    printf("%d ", array1D[k]);
}
printf("]\n");

return 0;
}

```

6. Execution & Verification Screenshots

```
kali@kali: ~/HDOC_Day28
$ mkdir HDOC_Day28
cd HDOC_Day28

(kali@kali)~/HDOC_Day28
$ nano day28.c

(kali@kali)~/HDOC_Day28
$ nano day28.2.c

(kali@kali)~/HDOC_Day28
$ gcc day28.c -o day28

(kali@kali)~/HDOC_Day28
$ ./day28
Enter positive number of rows: -2
Invalid input! Rows must be positive.
Enter positive number of rows: 2
Enter positive number of columns: 3

Enter elements column by column:
Element at Matrix[0][0]: 10
Element at Matrix[1][0]: 20
Element at Matrix[0][1]: 30
Element at Matrix[1][1]: 40
Element at Matrix[0][2]: 50
Element at Matrix[1][2]: 60

Matrix printed in Row-Major Order (using single loop & linear indexing):
10 30 50
20 40 60

(kali@kali)~/HDOC_Day28
$
```

```
(kali@kali)~/HDOC_Day28
$ gcc day28.2.c -o day28.2
./day28.2
Enter number of rows: 2
Enter number of columns: 2

Enter 2D Array Elements:
Matrix[0][0]: 1
Matrix[0][1]: 2
Matrix[1][0]: 3
Matrix[1][1]: 4

2D Array Matrix Representation:
1 2
3 4

Copied 1D Array Representation:
[ 1 2 3 4 ]

(kali@kali)~/HDOC_Day28
$
```